SF-HARNESS · ENGINEERING ASSIGNMENTS 01 AND 02

sf-harness, explained

What Assignment 01 asked for and what we built for it, then what Assignment 02 adds on top. Step by step, in plain words.

Abdul Basit · 5 November 2026 · repository **BasitJutt007/ai-harness**, branch **feat/assignment-02**

In one sentence: we built a program that lets an AI model change code only under strict, automatic supervision. In 01 the supervision was general (tests, API standards, safety). In 02 it is shaped to how I actually work, and three of my real backlog items go through it.

1 · The idea in two minutes
What a harness is, and the words used in this document

2 · Assignment 01: the brief
What was asked and how it is graded

3 · Assignment 01: what we built
Eight steps, from the engine to the pull request

4 · Assignment 02: the brief
Make it yours: three gates, three real items

5 · Assignment 02: what we built
Eight steps, from my habits to the mutation audit

6 · Where it stands, and how to run it
What is done, what waits for me, the commands

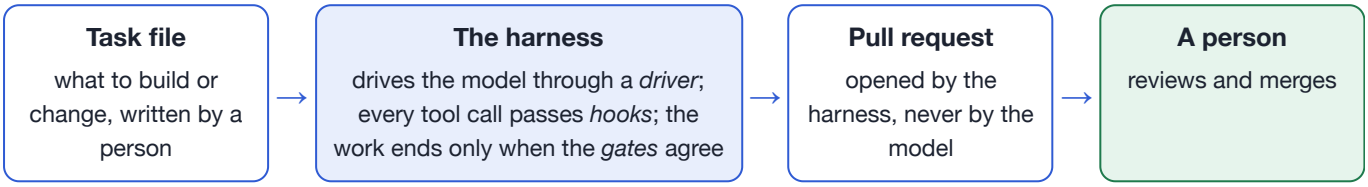
Everything here can be checked in the repository: `README.md`, `docs/workflow.md`, `docs/learnings.md`, `docs/contracts/`, `items/`, `runs/`.

PART 1

The idea in two minutes

An AI model can write code. Left alone, it can also cut corners: skip the tests, say "done" when it is not, edit files it should not touch, or spend far too much money. A **harness** is the program around the model that stops all of that.

The model only *proposes* actions ("read this file", "write that file", "run the tests"). The harness decides whether each action is allowed, runs it, and, at the end, decides with its own checks whether the work is really finished. Think of a driving-school car: the student drives, but the instructor has a second brake pedal and decides whether the lesson passed.



Words used in this document

Word	Meaning, in plain terms
Task file	A short file describing the job: what to build, which files may change, how it should behave.
Driver	The adapter that talks to one AI provider (Claude or OpenAI). Swapping drivers must not change anything else.
Tool	An action the model may ask for: read a file, write a file, search code, run tests, finish.
Hook	A guard that runs before or after every tool call and can block it, with a reason the model sees.
Check	A rule that reads the code and reports per file: pass, fail with `file:line`, or UNPROVEN when it cannot tell.
Gate	A final yes/no on the whole run (tests green, standards met, scope respected). "Done" means every gate passed.
Red, then green	A test is first seen failing (red) on the old code, then passing (green) on the new code: proof the change caused the fix.
Worktree	A separate copy of the repository for each run, so my own checkout is never touched.
Sandbox	An operating-system cage for code the model wrote: no network, no secrets, writes only to a temporary folder.
UNPROVEN	"Could not check". The harness never counts it as a pass.

The rule behind everything: nothing is green on anyone's word, not the model's and not mine. A pass needs evidence the harness produced itself; anything it could not check is reported as UNPROVEN.

ASSIGNMENT 01 the base

Assignment 01: the brief

Build a harness that governs TypeScript REST APIs. It must create a new API from a task file (**greenfield**) and safely change an existing API (**brownfield**). It must work with any AI provider, and it must prove the result meets our API standards.

What the graders check (the "01 rubric")

1 · Same task, both drivers. The identical task file runs on the Claude driver and the OpenAI driver, with the same hooks, gates and checks. The harness is provider-neutral.

2 · Token report. Every run records the tokens each turn used, compared with a baseline that sends everything every time.

3 · Standards checks. Errors as `application/problem+json`, REST conventions, strict TypeScript, and Zod validation at every request and response boundary.

4 · Extension tests. A new tool or check is added by dropping one file into `plugins/`, without editing the core.

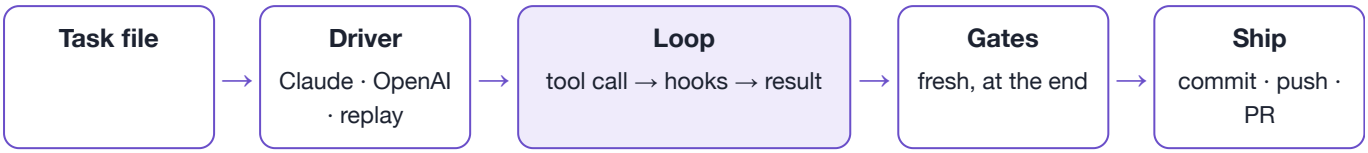
The two demo tasks

Task	Kind	What the model had to do
`users-api`	greenfield	Build a Users API from a template: create, list with cursor pagination, get, update, delete, with problem+json errors and idempotent creates.
`projects-change`	brownfield	Change an existing Projects API: add DELETE, and filter the list by status, without breaking any current client.

Both run against a separate public demo repository, `BasitJutt007/harness-demo`, so the harness never edits itself.

ASSIGNMENT 01 what we built

Assignment 01: what we built, step by step



1 The engine (`src/core/`)

A loop: send the task to the model, receive its tool calls, pass each through the hooks, run it, return the result, and repeat until the model calls `finish` and every gate agrees. The core never names a provider.

2 Drivers: one interface, any provider

`claude` and `openai` translate the same requests to each provider's API. A `scripted` driver replays recorded tool calls offline, so tests and demos need no keys.

3 Tools: what the model may do (15)

Read, write, edit, append and delete files; list and search code; outline a file; run tests; see the test map; fetch a standard's rules and check them; diff the API contract; plan; finish.

4 Hooks: a guard on every action (9)

For example: **path-guard** (only files inside the task's scope), **observed-red** (no source edit until a covering test was seen failing), **secret-guard**, **dependency-policy** and **test-preservation** (no deleting tests to get green).

5 Context on demand, and counting tokens

Instead of pasting the whole repository into every request, the model fetches what it needs and older turns are compacted. Measured against a real "send everything" run: **67% fewer input tokens per turn, 54% per run**.

6 Checks: the four API standards

`problem-json`, `rest-conventions`, `tsc-strict` and `zod-boundary` read the code and probe the running app. Anything they do not model (a redirect, a route registered in a loop) is UNPROVEN, never a pass.

7 Gates: the harness decides "done" (8)

Re-run fresh at the end: tests green; every changed file went red to green; standards at 100%; changes in scope; spec-coverage probes; **Contract Lock** (our own addition: no breaking API change); no secrets; no orphan files.

8 Sandbox and ship

Code the model wrote runs only in the OS sandbox. When every gate is green, the harness (never the model) commits, pushes a feature branch and opens the pull request.

Results. `users-api`: DONE in 14 turns, 33/33 tests, 18/18 spec units, standards 100% → harness-demo PR #10. `projects-change`: DONE in 21 turns, Contract Lock "2 additive changes", 20/20 tests → PR #11. Honest gap at the time: no Claude run reached DONE (there was no Anthropic key).

ASSIGNMENT 02 built on top of 01

Assignment 02: the brief

Make the harness yours. Encode how you actually work as hooks and checks, add three gates from the briefing that really block, run three real items from your own backlog through it, and write down what you learned. A renamed copy of someone else's harness scores zero.

30%	30%	15%	25%
Workflow fidelity: the gates look like me	Lessons as gates: three gates that block	Built on 01: 01 still passes	Backlog evidence: three items shipped

The three gates, and how the graders will try to skip them

Problem-statement gate. No source edit until `docs/contracts/<slug>.md` names the owner, the metric to move, WHEN/THEN scenarios and what is not included. Skip attempt: edit a file with no contract.

Human-yes gate. The stakeholder's yes must be a typed turn after the proposal, in their own words. Assistant text, tool output and hook output never count. The Stop is held until the yes exists. Skip attempt: echo "yes" from a tool and try to stop.

Outcome check. An item cannot close until its metric was read after the merge and written next to its value before. Skip attempt: close an item without reading the metric.

The deliverables

- The repository: 01 with 02 on top, visible in the history, one-command setup, a README naming the core directory.
- `docs/workflow.md`: how I work, mapped gate by gate.
- Three backlog items, each with its contract, recorded yes, red run, proof, harness-opened PR merged by a person, and before/after metric.
- A one-page learnings note: what it caught, what it got wrong, what I removed, what I would add next.

Due Sunday 8 November, 11:59 pm PKT. Tag the commit; no force-push afterwards.

ASSIGNMENT 02 what we built

Assignment 02: what we built, step by step

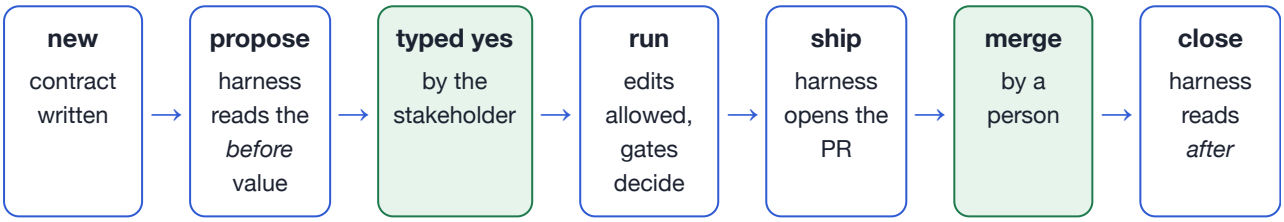
1 Studied how I really work

We mined my three company repositories (vp-business-backend, VendorPortal-React, VendorPortal-Admin: 503, 444 and 244 of my PRs), their review threads, CI and configs, and my own typed turns in 278 Claude Code sessions. The result is `docs/workflow.md`: how work reaches me (people, not tickets), where I lose time (waiting for my own go-ahead, large PRs, promoting by hand), what I repeat, and ten mistakes I have made more than once.

2 Every change starts with a contract

`docs/contracts/<slug>.md` names: Problem, Owner, Stakeholder, Repo, Base branch, Commit header, Metric (with before and after), Measure (the script that reads it), WHEN/THEN Scenarios, and Not included.

3 The three gates, as one lifecycle



- **Problem statement** (`problem-gate` hook): no edit while any field is missing; the refusal names each one. A run without a complete contract is refused before the first model call, so no tokens are spent.
- **Human yes** (`contract-gate` hook and `stakeholder-yes` gate): only a plain yes I type at a terminal, after the harness proposed this exact text. A piped reply, a "yes but…", a reply to an older proposal, a hand-written "Signed" line, and any "yes" in tool or model output are listed as "not counted". Until then every edit is refused and `finish` is held.
- **Outcome** (`harness item close`): only after a person (not a bot) merged the PR the harness opened, and the harness ran the same Measure script at the merge commit. No command-line flag can supply a value.

Each step is a line in `items/<slug>/ledger.jsonl`. Every line carries a hash of itself and of the line before, so a hand edit shows.

4 Taught it my stacks

Besides Express, it now profiles NestJS + Prisma + Jest (my backend) and React + Vite (my two frontends). Each check and gate declares which stacks it applies to, and the prompt tells the model its stack and the approved contract.

5 Turned my habits into checks

Every habit in `docs/workflow.md` points at a mechanism, and every mechanism points back at a habit:

What I keep getting wrong	What the harness now does
Coding before my own go-ahead	Contract + typed yes before any edit
Routes without the access check they need	`nest-route-guard`: admin routes need `@AdminOnly()`
Backend changes the frontends do not match	`route-lock`: no route removed or moved
A type check that checks zero files	`repo-typecheck` expands solution-style tsconfigs
Lint that lints no TypeScript	`repo-lint` reports such files as n/a, with the reason
A build broken only at deploy	`vite-build` builds the app, confined
Queries that read every row	`prisma-unbounded`: every findMany has a `take`
Commits and PRs that say nothing	The contract names the commit; the repository's commitlint and PR template are filled from it
Large PRs that bounce in review	`diff-budget`: at most 400 changed lines
Expensive models, surprise bills	Only gpt-5.6-luna and claude-haiku-4-5, at most $2 per run
Schema pushes that drift the shared DB	`prisma/` is unwritable; harness clones have no DB credentials
Odd payloads in tooling files	Config, lockfiles and non-TypeScript files are unwritable

6 Picked three real items and measured them first

Each defect was reproduced, and its metric script was attacked with "lazy fixes" that must not move the number before it was trusted. The harness read every before value itself.

Item	Repository	Metric (before)
`admin-only-sweep`	backend	admin-only routes any vendor account can call: **10**
`members-list-stale-after-add`	Admin app	Members lists left stale after adding a member: **1** of 3 checks fails
`signals-paged-merge-dedupe`	vendor app	duplicate rows after a refresh at page 2: **50**

I am the stakeholder for all three, because the team lead is on vacation. The gate still needs my own typed yes, and the learnings note says so.

7 Made sure nothing is green on anyone's word

The gates have paired tests (the bypass is refused, the honest twin passes). A mutation audit applies 31 broken variants to a shipped API and 51 mutations to items that pass each gate, and every one must be refused. `npm run verify` runs strict TypeScript and the full suite of tests.

8 Kept 01 working on top

The two 01 tasks became items too. They run against `harness-demo` branch `rubric-01` (the commit before the 01 PRs merged), on both drivers, with the allowed models.

PART 6

Where it stands, and how to run it

DONE

- The three gates, blocking, with tests and a mutation audit.
- Every habit in `docs/workflow.md` mapped to a built mechanism.
- NestJS and React + Vite support next to Express.
- Five contracts proposed, with before values read by the harness.
- 2063 tests and 86 audit cases green; README and learnings note updated; everything committed and pushed on `feat/assignment-02` (no PR opened).

WAITS FOR ME

1. Type my yes for each contract: `harness item reply <slug>`.
2. Merge the one-line jest prerequisite into the backend's `harness-test`.
3. Then the harness runs each item and opens its PR; I merge.
4. `harness item close` reads each after value.
5. Add what the runs showed to the learnings note; tag the commit on Sunday.

Commands

```
# once
npm run setup                                          # install, then a self-check (plugins,
sandbox, leaks)

# an item, start to finish
node bin/harness.mjs item new <slug>                     # contract template
node bin/harness.mjs item propose <slug> --repo <clone>  # reads the before value
node bin/harness.mjs item reply <slug>                   # the stakeholder types the yes
node bin/harness.mjs run tasks/<slug>.md --repo <clone> --driver openai --model gpt-5.6-luna --
ship
node bin/harness.mjs item close <slug> --repo <clone>    # after a person merged the PR
node bin/harness.mjs item status                        # every item's stage

# proof
npm run verify                                         # strict TypeScript + every test
npm run audit:mutations                                # 31 API + 51 gate mutations, all refused
```

What comes after 02

- An interactive terminal agent where my yes is a typed turn inside the session itself.
- Live NestJS probes (call each route with a vendor and an admin token) instead of reading decorators.
- A check that frontend API calls still match the backend's routes and payloads.
- A check that a Prisma schema change ships with its migration or backfill.